

Material de defensa — Primera entrega

Cloud Provider Analytics · Big Data · ITBA · 2C 2026 Versión 1.0 · 2026-09-20

Un apartado por sección del documento de diseño, con el **porqué** de cada decisión: argumento, evidencia que lo sostiene, alternativa descartada y preguntas previstas con su respuesta.

No forma parte del entregable. Es material interno para preparar la defensa oral y para que los cuatro integrantes sostengan un criterio común.

Cómo usarlo

Cada apartado tiene la misma estructura:

- 1. Tabla de decisiones tomadas y alternativas descartadas
- 2. Una sección por decisión: argumento · evidencia · "si preguntan..."
- 3. Preguntas incómodas previstas

Los cinco números que hay que saber de memoria

Dato	Valor	Dónde se usa
Eventos procesados por el streaming con watermark de 7 días	8,9%	§6, §9 — justifica la reconciliación
Usuarios con <code>last_login</code> anterior a <code>created_at</code>	35,1%	§3, §9 — justifica invariantes entre columnas
Costo perdido si se descartaran los registros defectuosos	4,6%	§9, §10 — justifica D-009
Distorsión por factura del FX ruidoso en USD	±15%	§8 — justifica D-007
Ratio de costo mediano <code>requests</code> / <code>cpu_hours</code>	87x	§11 — justifica MAD por serie

Índice

§	Sección
2	Problema, usuarios y objetivos
3	Las 5V
4	Inventario y perfil de fuentes
5	Arquitectura de alto nivel
6	Patrón arquitectónico
7	Matriz requisito-componente
8	Diseño del Data Lake
9	Flujos batch y streaming
10	Lógica MapReduce
11	Supuestos, riesgos y decisiones abiertas
12	Esfuerzo, roles y recursos

ewpage

Defensa · §2 Problema, usuarios y objetivos

Qué se decidió acá

#	Decisión	Alternativa descartada
1	Definir el problema como una tensión entre dos latencias	"Los datos están sucios y dispersos"
2	Caracterizar cada dominio por la decisión que toma	Describirlo por el reporte que consume
3	Numerar las preguntas P1-P7 y trazarlas a las consultas obligatorias	Listar preguntas sueltas
4	Objetivos con umbral verificable	Objetivos cualitativos
5	Declarar P7 (ML) como deseable desde el inicio	Comprometerla y ver después

1. Por qué el problema son dos latencias

El argumento. Decir "los datos llegan crudos y dispersos" describe el síntoma, no el problema. El problema es que **FinOps necesita intervenir un consumo anómalo en minutos y la facturación se consolida una vez por mes**. Resolver una sola de las dos deja la mitad del negocio sin respuesta.

Por qué importa esta formulación. Es el argumento que después sostiene toda la §6. Si el problema fuera "datos sucios", la solución sería un ETL y no habría discusión de patrón. Planteado como tensión de latencias, la elección de Lambda, Kappa o híbrido deja de ser una preferencia y pasa a ser una consecuencia.

Si preguntan "¿no están forzando el caso para justificar streaming?". Al revés: es el argumento que nos lleva a **no** usar streaming en los maestros. La tensión justifica rutas separadas, no streaming en todo.

2. Por qué caracterizamos los dominios por la decisión que toman

El argumento. La consigna define tres dominios: FinOps, Soporte y Producto. Lo que agregamos son dos columnas: **qué decisión toma cada uno y con qué latencia la necesita**.

Por qué esas dos columnas. Son las que determinan si la ruta es batch o streaming. El nombre del dominio no determina nada: FinOps tiene una necesidad de minutos (anomalías) y otra mensual (revenue), y por eso aparece en las dos rutas.

Si preguntan "¿de dónde sacaron las latencias?". Son un supuesto nuestro, derivado de la decisión que habilita cada dominio: una anomalía detectada después del cierre ya se facturó. Está declarado como tal, y los umbrales concretos (O1: 5 minutos) son ajustables.

3. Los objetivos con umbral

El argumento en una línea. Un objetivo sin umbral no es verificable.

Ejemplo del contraste. "Buena frescura de datos" no se puede evaluar. "Latencia evento → mart consultable ≤ 5 min, verificado en la segunda entrega" sí.

Lo que además hace. Cada objetivo declara **en qué instancia se verifica**, lo que convierte la tabla en el insumo de los criterios de aceptación de las entregas siguientes.

Si preguntan "¿los umbrales son alcanzables?". O1 y O6 son estimaciones razonables para un MVP, no mediciones. Se ajustarán con la primera medición real en noviembre, y ese ajuste se registrará.

4. Por qué P7 se declaró deseable desde el inicio

La evidencia. 20 de 80 organizaciones nunca fueron encuestadas y el 25,4% de los tickets no tiene CSAT. Para un modelo supervisado de riesgo de abandono, la señal es débil.

La decisión. Declararla deseable en la §2.3, no obligatoria.

Por qué ahora y no en noviembre. La consigna admite "una alternativa equivalente acordada" para el componente analítico. Plantearlo en la primera entrega abre la conversación con la cátedra con siete semanas de margen; plantearlo en diciembre es llegar con un problema en lugar de con una propuesta.

ewpage

Defensa · §3 Las 5V

Qué se decidió acá

#	Decisión	Alternativa descartada
1	Abrir admitiendo que 2,46 MB no justifican Big Data	Presentar el dataset como si fuera grande
2	Justificar por réplica reducida + dominancia de otras V	Justificar por volumen
3	Cerrar cada V con la decisión de arquitectura que fuerza	Describir las V en abstracto
4	Declarar Veracidad como la V dominante	Tratar las cinco como equivalentes

1. La decisión más arriesgada del documento

Lo que hacemos. La §3.1 empieza diciendo que **el dataset pesa 2,46 MB y que ese volumen no justifica Big Data**. Se procesa con pandas en una notebook y sobra.

Por qué es la jugada correcta. La consigna pide *justificar la necesidad*, no afirmarla. Y la Clase 03 dice textualmente que usar Spark "por moda" produce notebooks lentos, costosos y difíciles de depurar, y que la decisión técnica madura empieza por elegir bien la herramienta. Un documento que sostenga que 7.200 eventos son "volumen" se cae con una sola pregunta.

La justificación real, en dos movimientos:

- El dataset es una réplica reducida, no la carga.** Conserva todas las características cualitativas del caso —variedad, evolución de esquema, tipos ambiguos, llegada tardía, anomalías— y reduce solo el volumen. Diseñar para el tamaño de la muestra obliga a rehacer al primer crecimiento.
- El volumen es una de cinco V, y no la dominante.** Lo que exige un motor distribuido acá es la combinación de velocidad con desorden, variedad estructural y veracidad comprometida.

El número que respalda el punto 1. Los eventos provistos equivalen a **0,0125 eventos por recurso-hora**: un 1,25% de lo que emitiría una telemetría que reportara una sola métrica por recurso y por hora. La muestra está deliberadamente diezmada, y eso es verificable.

Si preguntan "¿entonces esto no es un proyecto de Big Data?". Es un proyecto de Big Data dimensionado sobre una muestra. La arquitectura se diseña para el caso —que escala a más de mil millones de eventos anuales en el escenario mediano— y se demuestra sobre la réplica. Confundir las dos cosas es el error que estamos evitando.

2. Por qué cada V cierra con una decisión

El criterio. Una V que no llega a un componente es retórica. Cada subsección termina con "decisión que fuerza", y esas decisiones son las que después aparecen en la matriz de la §7.

El efecto. Cuando en la defensa pregunten por qué hay watermark, la respuesta no es "porque Structured Streaming lo tiene": es que la Velocidad de este caso se manifiesta como desorden estructural, y el watermark es la respuesta a eso.

3. Por qué Veracidad es la V dominante

La evidencia. Diez riesgos de calidad documentados, de los cuales cuatro no los detecta ningún control de completitud. El perfilado dedicó más esfuerzo a veracidad que a las otras cuatro V juntas, y el diseño lo refleja: quarantine, invariantes entre columnas, flags, y una decisión entera (D-009) sobre el alcance del rechazo.

Si preguntan "¿por qué no Volumen?". Porque el volumen de la muestra es trivial y el del caso es un problema resuelto por particionado estándar. La veracidad, en cambio, requiere decisiones de diseño específicas de este dataset.

ewpage

Defensa · §4 Inventario y perfil de fuentes

Qué se decidió acá

#	Decisión	Alternativa descartada
1	Perfilar con PySpark, no con pandas	pandas, más rápido de escribir
2	Cardinalidad exacta, no aproximada	<code>approx_count_distinct</code>
3	Derivar los rangos válidos del dominio observado	Asumir escalas estándar
4	Documentar las dos correcciones en lugar de borrarlas	Presentar solo el resultado final
5	Distinguir nulos semánticos de defectos	Contar todos los nulos como problemas

1. Por qué perfilamos con PySpark

El argumento. A este volumen pandas sería más rápido de escribir. Pero el script de perfilado es también la primera pieza del pipeline: fija el patrón de lectura con esquema explícito, la configuración de sesión y las convenciones que usarán los jobs siguientes.

El beneficio colateral. Verificó que el entorno funciona: PySpark 4.2.0 sobre Java 21, la misma versión que la Clase 03 menciona como actual.

Si preguntan "¿no es sobredimensionado?". Para el perfilado sí, y lo diríamos igual. La alternativa era escribir el mismo trabajo dos veces.

2. Las dos correcciones, y por qué están en el documento

Esto es lo que más conviene saber contar, porque muestra método.

Corrección 1 — cardinalidad. `approx_count_distinct` opera sobre HyperLogLog y a esta escala reportaba **83 org_id distintos sobre 80 filas**. Se cambió a `count_distinct`. Registrado como D-001, con la nota de que a volumen productivo la decisión se revierte.

Corrección 2 — la escala del NPS. La primera pasada evaluó `nps_score` contra la escala 0–10 de una respuesta individual de encuesta y marcó **59 de 80 registros como anómalos**. La distribución real (mín. −38, máx. 101, mediana 14) corresponde a la **métrica NPS agregada, cuya escala es −100 a +100**. Con la regla correcta hay **una sola** violación.

Por qué las dejamos escritas. Porque el criterio que sale de la segunda —los rangos válidos se derivan del dominio, no se asumen— es el que después evita el error grande: escribir la regla de `genai_tokens` como "v2 implica tokens" en lugar de `v2 AND service = 'genai'` habría producido **251 falsos positivos sobre 275 eventos v2**.

La frase que resume. Una regla de calidad mal calibrada no genera falsos positivos: genera desconfianza en todo el control de calidad. Una regla que grita todo el tiempo se termina ignorando.

Si preguntan "¿por qué muestran sus propios errores?". Porque el criterio vale más que el resultado. Cualquiera puede presentar una tabla de nulos; lo que distingue al perfilado es haber validado las reglas contra el dominio antes de confiar en ellas.

3. La distinción entre nulo semántico y defecto

El criterio. Un nulo es **semántico** si existe una lectura de negocio que lo explique; es **defecto** si viola una regla declarada.

Aplicado:

- 240 `resolved_at` nulos → tickets abiertos → **estado válido**, promueve.
- 254 `csat` nulos → encuesta no respondida → **estado válido**, promueve.
- 139 `last_login` nulos → usuario que nunca ingresó → **estado válido**.
- 17 `unit` nulos con `value` presente → **defecto**.

Por qué importa. Si los 240 `resolved_at` fueran a quarantine, el mart de Soporte no podría contar tickets abiertos. La consigna pide SLA y volumen por severidad: los abiertos son parte de la respuesta, no ruido.

Si preguntan "¿y quién decide qué es semántico?". Se documenta por campo en el diccionario de datos, que es propiedad del rol de arquitectura y gobierno. No es criterio de quien escribe el job.

4. El hallazgo que más pesó en el diseño

La fragmentación de eventos es aleatoria, no temporal. El archivo perfilado cubre 59 días y sus registros no están ordenados: el primero es del 18/08 y el segundo del 12/08.

Las tres consecuencias directas, que después aparecen en §6, §8 y §9:

- La llegada tardía es la norma, no la excepción de red.
- La deduplicación por `event_id` debe ser inter-lote con estado.
- Particionar Bronze por fecha de evento generaría small files en cada micro-lote.

Si preguntan "¿cómo lo detectaron?". Mirando el orden de los registros y midiendo el span temporal por archivo, no solo el rango global del dataset. Es la clase de cosa que un `describe()` no muestra.

Defensa · §5 Arquitectura de alto nivel

Material de preparación para la instancia oral. No se entrega como parte del documento de diseño: sirve para sostener las decisiones cuando nos pregunten.

Las decisiones que se tomaron acá

#	Decisión	Alternativa descartada
1	Dos rutas con latencias distintas	Ruta única
2	Convergencia en Silver, no en Bronze	Unificar el modelo desde la ingesta
3	Capacidades transversales, no una etapa de calidad	Un paso de validación entre zonas
4	Quarantine como zona propia	Columna de flag dentro de cada tabla
5	Colab / <code>local[*]</code> declarado como supuesto	Diagramar un clúster que no vamos a tener

1. Por qué dos rutas y no una

El argumento. En la §2.1 el problema se definió como una tensión entre dos latencias incompatibles: FinOps necesita detectar una anomalía de costo en minutos, mientras que la facturación se consolida una vez por mes. Una ruta única obliga a elegir: o se procesa todo en streaming y se paga complejidad operativa en fuentes que cambian una vez al mes, o se procesa todo en batch y se pierde la capacidad de intervenir antes del cierre.

La evidencia. `billing_monthly.month` tiene exactamente 3 valores distintos en 240 filas. Es una fuente que se actualiza mensualmente. Tratarla como stream no reduce ninguna latencia que a alguien le importe.

Si preguntan "¿por qué no todo en streaming?". Porque la complejidad tiene que estar justificada por un requisito. Structured Streaming sobre los maestros implicaría checkpoints, estado y watermarks para procesar 80 filas que cambian a diario. La Clase 03 lo plantea directamente: usar Spark "por moda" genera pipelines lentos, costosos y difíciles de depurar. El mismo criterio aplica dentro de Spark.

Si preguntan "¿por qué no todo en batch?". Porque incumple O1. Un lote nocturno significa que una anomalía de costo se detecta hasta 24 horas después. En FinOps eso equivale a explicarla, no a evitarla.

2. Por qué las rutas convergen en Silver y no antes

El argumento. Bronze tiene una responsabilidad específica: preservar el grano y el formato de la fuente con tipado explícito y trazabilidad. Si ahí se unificara el modelo, se perdería la correspondencia uno a uno con el archivo de origen y con ella la capacidad de reprocesar una fuente sin tocar las demás.

La evidencia. Las fuentes tienen granos genuinamente distintos: organización, usuario, recurso, ticket, interacción, factura mensual y evento. No existe un grano común antes de conformar.

Si preguntan "¿no es un paso de más?". Silver es el único lugar donde se puede resolver la compatibilidad v1/v2 de forma centralizada. Si los eventos se enriquecieran en Bronze, la promoción de v1 al esquema ampliado quedaría mezclada con la ingesta, y cada cambio de esquema futuro obligaría a tocar el lector.

3. Por qué la calidad es transversal y no una etapa

El argumento. Una etapa de validación ubicada entre Bronze y Silver solo valida lo que atraviesa ese punto. Los problemas que documentó el perfilado aparecen en momentos distintos: los duplicados de clave natural son un problema de ingesta, las invariantes entre columnas son un problema de conformación, y las anomalías de costo solo son detectables después de calcular la distribución.

La evidencia. 17 `ticket_id` duplicados (ingesta), 232 usuarios con `last_login` anterior a `created_at` (conformación), 6 eventos con `|z| > 3` (post-agregación). Tres controles, tres momentos distintos.

Si preguntan "¿dónde se aplica concretamente?". En cada frontera entre zonas, con reglas propias de esa frontera. Se especifica en la §9.

4. Por qué quarantine es una zona y no una columna

El argumento. La consigna exige "quarantine en Parquet" explícitamente. Pero además hay una razón de diseño: si el registro inválido queda como flag dentro de la tabla principal, toda consulta posterior tiene que acordarse de filtrarlo. Es cuestión de tiempo hasta que una no lo haga.

El matiz que hay que saber defender. No todo lo que falta va a quarantine. Los 240 `resolved_at` nulos son tickets abiertos y los 254 `csat` nulos son encuestas no respondidas: son **estado válido**, no defectos. Mandarlos a quarantine rompería el mart de Soporte, que necesita contar tickets abiertos. En cambio los 17 `unit` nulos con `value` presente sí son defectos.

Si preguntan "¿cuál es el criterio?". Un nulo es semántico si existe una lectura de negocio que lo explique; es defecto si viola una regla declarada. La distinción se documenta por campo en el diccionario de datos.

5. Por qué declaramos Colab como supuesto en lugar de dibujar un clúster

El argumento. El criterio de aceptación de la 2.^a entrega dice que el diagrama debe representar lo implementado "y no componentes meramente aspiracionales". Dibujar YARN, S3 y un clúster de nodos en el v1 cuando vamos a correr en `local[*]` sería precisamente eso.

Lo que sí se sostiene. La API, el plan de ejecución, el DAG y el particionado son los mismos conceptos en local y en clúster — la Clase 03 lo plantea así. El diseño de particiones se justifica contra el caso productivo dimensionado en la §3.2, no contra el runtime de la demo.

Si preguntan "¿entonces esto no es Big Data?". Ver el documento de defensa de la §3. La respuesta corta: el dataset es una réplica reducida que conserva las características cualitativas del caso y solo diezma el volumen; la arquitectura se diseña para el caso, no para la muestra.

Si preguntan "¿qué cambia en producción?". Landing, Bronze, Silver y Gold pasan a almacenamiento de objetos y el motor corre sobre YARN o Kubernetes. Las zonas, los flujos y el modelo de serving no cambian. Esa portabilidad es deliberada y es la razón por la que el entorno está aislado en una sola subsección.

Preguntas incómodas previstas

"¿Por qué Cassandra y no un data warehouse?" Es decisión de la consigna, pero se sostiene sola: el patrón de consulta está definido de antemano (P1 a P5), las lecturas son por clave de partición conocida —organización y rango de fechas— y el volumen de escritura es alto y continuo. Es exactamente el caso de uso de un modelo query-first. Lo que Cassandra no da es consulta ad-hoc, y por eso Gold en Parquet se conserva como capa de análisis exploratorio.

"¿Qué pasa si se cae el streaming?" El checkpoint permite retomar desde el último offset confirmado. Como la deduplicación es por `event_id` y los upserts en Cassandra son idempotentes, el reprocesamiento no duplica. Es el objetivo O4 y se demuestra con conteos antes/después.

"¿Por qué Parquet y no Delta o Iceberg?" La consigna fija Parquet. Si preguntan qué aportarían: transacciones ACID, evolución de esquema gestionada y time travel — justamente los tres problemas que acá se resuelven a mano. Se menciona como próximo paso en la §12, no como carencia del diseño.

"El diagrama muestra Quarantine colgando de Bronze y Silver. ¿Por qué no de Gold?" Porque en Gold ya no hay registros inválidos: lo que llega ahí pasó todos los controles. Lo que sí hay en Gold son **flags de anomalía**, que son otra cosa: un costo negativo de −19,82 puede ser una nota de crédito legítima y

se marca, no se rechaza.

Defensa · §6 Patrón arquitectónico

La decisión en una frase

Híbrido: rutas segmentadas por fuente según la latencia que su decisión de negocio exige, más un job batch de reconciliación que corrige lo que el watermark del streaming descarta.

#	Decisión	Alternativa descartada
1	Híbrido, no Kappa	Todas las fuentes como stream
2	Híbrido, no Lambda canónico	Recomputación duplicada de todo el dato
3	Reconciliación batch sobre la ruta de eventos	Confiar solo en el watermark
4	Watermark corto + corrección posterior	Watermark largo que cubra el span completo

1. El punto fuerte del argumento: por qué no es Lambda

Esta es la parte que conviene tener afilada, porque es donde se nota si el patrón se eligió o se copió.

Lambda canónico procesa **los mismos datos** por dos caminos: una capa batch que recomputa la vista completa desde el dato crudo, y una capa de velocidad que produce una vista incremental aproximada. La capa de serving **fusiona ambas al momento de la consulta**. El costo que define al patrón es mantener dos implementaciones de la misma lógica.

Lo nuestro no es eso. Los eventos van por streaming y los maestros por batch: fuentes distintas por caminos distintos. No hay dos vistas del mismo dato ni fusión en la consulta.

Si preguntan "¿entonces no es Lambda?". Correcto, y es deliberado. La mayoría de las soluciones declara Lambda cuando en realidad tiene batch y streaming conviviendo, que no es lo mismo. Lo que sí tomamos de Lambda es un elemento acotado y justificado: la reconciliación.

2. Por qué descartamos Kappa

El argumento. Kappa tiene una sola base de código, que es su virtud real. Pero exige modelar todo como stream.

La evidencia. `billing_monthly.month` tiene 3 valores distintos en 240 filas: se consolida una vez por mes. `customers_orgs` son 80 registros. Poner checkpoints, estado y watermarks sobre eso agrega complejidad operativa sin reducir ninguna latencia que a alguien le importe.

El segundo argumento, más técnico. Kappa depende de poder reproducir el log completo para hacer backfill. En Colab no hay bus de eventos persistente: el re-stream sería releer archivos, que es batch con otro nombre. El patrón perdería su propiedad definitoria.

Si preguntan "¿cuándo elegirían Kappa?". Si las fuentes llegaran por un log persistente tipo Kafka y el equipo pagara el costo operativo de una plataforma de streaming real. Ahí Kappa eliminaría la duplicación que hoy aceptamos.

3. El núcleo: por qué hace falta la reconciliación

Esta es la parte del diseño que sale directamente del perfilado y que más difícil es de improvisar. Conviene poder explicarla sin leer.

El hallazgo. Cada archivo de eventos cubre **59 días y no está ordenado cronológicamente**. El primer registro del archivo es del 18/08 y el segundo del 12/08. La fragmentación es aleatoria, no temporal.

El problema que genera. Structured Streaming descarta los eventos que llegan más tarde que el watermark. Y acá hay una tensión que no tiene salida limpia:

- Watermark **largo** (59 días, cubriendo el span observado): habría que sostener dos meses de estado y ninguna ventana se finalizaría hasta dos meses después. Incumple O1.
- Watermark **corto** (días): cumple O1, pero descarta eventos tardíos.

La consecuencia que hay que admitir. La ruta de streaming **va a producir agregados incompletos**. No por un bug: por la naturaleza de la fuente.

La solución. Un job batch que recomputa los marts desde Bronze, donde los eventos tardíos sí quedaron persistidos aunque la agregación en streaming los haya ignorado. Streaming da frescura, batch da corrección.

Si preguntan "¿por qué no simplemente un watermark más largo?". Porque el costo es estado proporcional a la ventana y latencia de finalización igual a la ventana. Con 59 días, el mart "fresco" tendría dos meses de retraso, que es exactamente lo contrario de lo que O1 pide.

Si preguntan "¿no pierden datos igual?". No: Bronze persiste **todos** los eventos, tardíos incluidos. Lo que el watermark descarta es su participación en la agregación en tiempo real, no el dato. La reconciliación los recupera.

4. La honestidad que conviene mostrar

Hay dos cosas que declaramos en el documento y que es mejor decir antes de que nos las pregunten.

El patrón tiene costos, y están escritos (§6.7): lógica duplicada acotada a los marts de eventos, ventana de inconsistencia entre corridas, y dos rutas que mantener. Un patrón cuyo costo no se declara no está justificado, está defendido.

La verificación está pendiente (A-07). El perfilado se hizo sobre **un solo archivo** de los ~20. Si el dataset completo mostrara que los eventos sí llegan ordenados, la reconciliación dejaría de ser necesaria y el patrón se reduciría a segmentación simple. Está anotado como decisión abierta con fecha objetivo.

Decir esto es más fuerte que ocultarlo: muestra que la conclusión está atada a la evidencia disponible y que sabemos cuál evidencia la cambiaría.

Preguntas incómodas previstas

"¿La reconciliación no convierte esto en Lambda después de todo?" En parte, y es intencional. La diferencia con Lambda canónico es el alcance: acá la recomputación se aplica **solo a los marts derivados de eventos**, no a todo el dato, y la fusión no ocurre en la consulta sino por sobrescritura de la partición. Es Lambda aplicado donde hace falta en lugar de como principio general.

"¿Cada cuánto corre la reconciliación?" Se define en la §9 junto con el watermark (A-05). El criterio: suficientemente seguido para que la ventana de inconsistencia sea tolerable, suficientemente espaciado para no recomputar de más. Diaria es el punto de partida razonable.

"¿Cómo evitan que la reconciliación duplique registros?" Recalcula la partición completa desde Bronze y la sobrescribe. Es idempotente por construcción, sin necesidad de lógica de merge. Sostiene O4 sin trabajo adicional.

"¿No es un desperdicio recomputar todo?" A este volumen, no. A escala productiva se acotaría a las particiones tocadas por eventos tardíos en la última corrida, que es información que Bronze tiene vía `ingest_ts`. Está anotado como optimización, no como parte del MVP.

"Si streaming y batch dan resultados distintos, ¿cuál es el correcto?" El de la reconciliación, siempre. El mart expone la marca de la última corrida de reconciliación para que el consumo sepa si está mirando un valor provisorio o consolidado.

Defensa · §7 Matriz requisito-componente

Qué se decidió acá

#	Decisión	Alternativa descartada
1	Cuatro matrices encadenadas, no una sola tabla	Una matriz única requisito × componente
2	Trazar contra los 12 requisitos de la §4.4 de la consigna	Trazar solo contra nuestros propios objetivos
3	Declarar los huecos de cobertura en una sección propia	Mostrar cobertura total
4	Anticipar que P1 y P2 necesitan tablas de serving distintas	Asumir un mart = una tabla

1. Por qué cuatro matrices y no una

El argumento. Una matriz única requisito × componente colapsa cuatro preguntas distintas en una sola grilla: qué exige la teoría (5V), qué pide el negocio (P1-P7), qué nos comprometimos a lograr (O1-O8) y qué exige la cátedra (§4.4). Cada una tiene una audiencia y una forma de verificarse.

La cadena que arma. Va de lo conceptual a lo verificable: 5V → decisión → componente → evidencia. Una V que no llega a un componente es retórica; un componente que no viene de una V es un capricho.

Si preguntan "¿no es redundante?". Hay solapamiento deliberado: el mismo componente aparece en varias matrices por razones distintas. Structured Streaming aparece por Velocidad (5V), por O1 (frescura) y por el requisito de ingesta streaming. Que converjan tres justificaciones sobre el mismo componente es señal de que está bien elegido.

2. Por qué trazamos contra los 12 requisitos de la consigna

El argumento. Es la matriz más aburrida y la más útil en la corrección. La §4.4 de la consigna enumera doce capacidades obligatorias del proyecto completo. Si alguna no tiene componente asignado, es un agujero que aparece en noviembre, no ahora.

Lo que además hace. Asigna cada requisito a una **instancia de entrega**. Eso convierte la matriz en el insumo directo del backlog de la segunda entrega y de la estimación de esfuerzo de la §12.

3. El hallazgo de modelado: P1 y P2 leen el mismo mart pero necesitan tablas distintas

Es el punto técnicamente más fino de la sección y conviene tenerlo claro porque anticipa un problema de la entrega final.

El problema. P1 pide costos y requests diarios por organización y servicio en un rango de fechas. P2 pide el Top-N de servicios por costo acumulado en los últimos 14 días. Las dos salen del mismo mart Gold, `org_daily_usage_by_service`.

Por qué no alcanza una tabla. En Cassandra el orden de los resultados lo fija la **clustering key**, no la consulta. P1 necesita ordenar por fecha para filtrar un rango; P2 necesita ordenar por costo descendente para cortar en N. Una sola tabla obliga a traer todas las filas y ordenar en el cliente, que es exactamente lo que el modelado query-first existe para evitar.

La consecuencia. Un mart Gold puede derivar en **más de una tabla de serving**. La desnormalización no es un defecto del modelo: es el modelo.

Si preguntan "¿no duplica datos?". Sí, y es correcto. En Cassandra el costo de almacenamiento es barato y el de un escaneo no acotado es caro. Se escribe dos veces para leer una vez bien.

4. Por qué declaramos los huecos (§7.6)

El argumento. Una matriz que muestra 100% de cobertura invita a buscar dónde está la mentira. Declarar cuatro huecos con su motivo hace creíble el resto.

Los cuatro, y cómo defender cada uno:

- P7 sin mart.** 20 de 80 organizaciones sin encuesta NPS y 25,4% de tickets sin CSAT. La señal es débil para un modelo supervisado. La consigna admite explícitamente "una alternativa equivalente acordada", y preferimos plantearlo ahora que descubrirlo en diciembre.
- Performance no verificable.** En `local[*]` el paralelismo es simulado. El particionado se justifica contra el caso dimensionado en la §3.2. Lo que sí se puede evidenciar son tamaños, rutas y cantidad de archivos.
- Retención no ejercitable.** El dataset cubre 4 meses; una política a 12 o 24 meses no se demuestra. Se especifica igual, porque la consigna pide la definición, no su ejecución.
- Reconciliación pendiente de validar (A-07).** Depende de perfilar los ~20 archivos completos.

Si preguntan "¿entonces no cumplen con todo?". Cumplimos con los doce requisitos obligatorios: los cuatro huecos son limitaciones de verificación o alcances declarados como deseables, no requisitos sin asignar. La diferencia está en las columnas de la §7.5.

Preguntas incómodas previstas

"**¿Por qué P6 (anomalías) sale de un mart y no de los eventos crudos?**" Porque una anomalía se define contra un comportamiento histórico, y eso exige la serie diaria ya agregada. `cost_anomaly_mart` deriva de `org_daily_usage_by_service`, no de Silver. Es una dependencia entre marts y está declarada en la §7.3.

"**El mismo componente aparece en tres matrices. ¿Cuál manda?**" Ninguna: son vistas distintas del mismo diseño. Si hubiera contradicción entre ellas, sería un error del diseño y no de las matrices — que es justamente para lo que sirven.

"**¿Cómo saben que los granos de los marts son los correctos?**" Los granos de los cinco marts obligatorios los fija la §7.3 de la consigna. Lo que aportamos es la trazabilidad: qué fuentes los alimentan, por qué ruta llegan y qué pregunta responden.

"**¿Qué pasa si una pregunta nueva no entra en ningún mart?**" Se agrega una tabla de serving, no se fuerza una consulta sobre una existente. Es la

contracara de query-first: el modelo se extiende por pregunta. Gold en Parquet queda como capa de exploración ad-hoc para descubrir esas preguntas antes de modelarlas.

Defensa · §8 Diseño del Data Lake

Qué se decidió acá

#	Decisión	Alternativa descartada
1	Bronze particiona por fecha de ingesta ; Silver y Gold por fecha de evento	Particionar todo por fecha de evento y compactar después
2	<code>service</code> como segunda clave; <code>org_id</code> nunca	Particionar por organización
3	<code>nps_surveys</code> es fuente de verdad, pero se conservan ambos valores	Sobrescribir el CRM con la última encuesta
4	Forzar tasa 1,0 para facturas en USD	Confiar en el campo tal como viene
5	Un mart Gold puede originar más de una tabla de serving	Una tabla por mart

1. El cambio de eje de partición: la decisión central de la sección

Es lo más original del diseño y sale directamente del perfilado. Conviene poder explicarlo sin leer.

El problema. Los archivos de eventos cubren 59 días sin orden cronológico. Si Bronze se particionara por **fecha del evento**, cada micro-lote escribiría en casi todas las particiones del histórico: decenas de archivos diminutos por corrida, multiplicados por cada micro-lote. Es small files en su forma más aguda.

La decisión. Bronze particiona por `ingest_date`: cada micro-lote escribe en **una sola** partición. Silver y Gold particionan por `event_date`.

La formulación que conviene usar. Bronze responde "*qué llegó y cuándo*" — semántica de auditoría. Silver responde "*qué pasó y cuándo*" — semántica analítica. El cambio de eje es lo que separa auditoría de análisis, no un truco de performance.

El beneficio que no era obvio. El job de reconciliación de la §6 puede leer solo las particiones de Bronze con `ingest_date` reciente para saber **qué fechas de evento fueron tocadas por datos tardíos**, y recomputar únicamente esas particiones de Gold. Sin ese eje, la reconciliación tendría que recomputar todo el histórico o rastrear los tardíos fila por fila.

Si preguntan "¿por qué no compactar después?". Porque agrega un job de compactación para resolver un problema que el cambio de eje evita de entrada. Menos piezas móviles.

2. Por qué `service` sí y `org_id` no

`service`: cardinalidad 6, estable, conocida de antemano, y P1 y P2 filtran o agrupan por él. Es el caso de libro para una clave de partición.

`org_id`: 80 valores hoy, miles en el caso dimensionado de la §3.2. Produciría particiones diminutas y un directorio inmanejable. La Clase 03 lo plantea directo: no conviene particionar por columnas de altísima cardinalidad.

Si preguntan "¿pero las consultas filtran por organización?". Sí, y por eso `org_id` **es** clave de partición en Cassandra, que es donde esa consulta se ejecuta. En el lago el criterio es el tamaño de bloque; en el serving es el patrón de acceso. Son dos sistemas con dos criterios distintos, y confundirlos es un error frecuente.

3. La honestidad sobre la muestra

Lo que declaramos. Con 7.200 eventos en 60 días hay ~120 eventos diarios. Particionar por fecha y servicio da particiones de unas pocas decenas de filas: **a escala de la muestra el esquema está sobre-particionado**. Se compensa con `coalesce(1)` al escribir.

Por qué lo declaramos en lugar de ocultarlo. Porque es verificable con un `ls` sobre el directorio de salida en la segunda entrega. Mejor explicarlo ahora en los términos correctos —el esquema se diseña para el caso dimensionado, no para la demo— que defenderlo cuando lo señalen.

Si preguntan "¿entonces el particionado está mal?". Está bien para el caso y sobra para la muestra. Lo que estaría mal es diseñar para 7.200 filas y tener que rehacerlo al primer crecimiento.

4. NPS: por qué conservamos las dos columnas

El argumento. `nps_surveys` es el hecho con fecha, y por eso es fuente de verdad. Pero **20 de las 80 organizaciones nunca fueron encuestadas**: si se eliminara `customers_orgs.nps_score`, esas 20 quedarían sin NPS.

La decisión. Silver conserva ambos con nombres distintos — `nps_score_crm` y `nps_score_last_survey` — más la fecha de la encuesta. No se sobrescribe uno con otro: se expone la discrepancia y Gold elige, marcando el origen.

Si preguntan "¿no es duplicar información?". Son dos hechos distintos: lo que dice el CRM y lo que dijo la última encuesta. Colapsarlos en una columna pierde la capacidad de detectar que el CRM está desactualizado, que es información útil para Customer Success.

5. FX: el error que se cancela en el total

Vale la pena tenerlo afilado porque el razonamiento es contraintuitivo.

El hallazgo. Las tasas de ARS (mediana 0,00150) y EUR (mediana 1,10467) son plausibles. Las 160 filas en USD tienen tasas entre **0,85463 y 1,11791** en lugar de 1,0. Convertir dólares a dólares tiene tasa 1 por definición: eso es ruido, no una tasa.

El dato que sorprende. En el agregado el ruido se compensa: la diferencia de revenue total entre aplicar el FX siempre y forzar 1,0 en USD es del **0,1%**. Una validación global no lo detectaría nunca.

Por qué corregirlo igual. Por factura la distorsión llega a **−15% / +12%**, y los marts de FinOps reportan por organización y por mes, no en total. **Un error que se cancela en el total sigue siendo un error en cada fila** — y cada fila es la factura de un cliente.

Si preguntan "¿y si la tasa es correcta y la moneda está mal etiquetada?". Sería la hipótesis alternativa, pero no se sostiene: las tasas de USD se distribuyen alrededor de 1,0 con dispersión simétrica, que es la firma de un ruido inyectado. Si la etiqueta estuviera mal, las tasas se parecerían a las de ARS o EUR, y no es el caso. Igual dejamos la fila marcada con `fx_usd_ruidoso` para que la corrección sea auditable y reversible.

Preguntas incómodas previstas

"¿Por qué Quarantine es una zona y no una columna?" La consigna lo pide en Parquet, pero además: si el inválido queda como flag dentro de la tabla, toda consulta posterior tiene que acordarse de filtrarlo. Es cuestión de tiempo hasta que una no lo haga.

"¿Cómo distinguen un nulo que es defecto de uno que no?" Un nulo es **semántico** si existe una lectura de negocio que lo explique; es **defecto** si viola una regla declarada. Los 240 `resolved_at` nulos son tickets abiertos y promueven. Los 17 `unit` nulos con `value` presente son defectos y no promueven. El criterio se documenta por campo en el diccionario.

"La retención dice 24 meses pero tienen 4 meses de datos." Correcto, y está declarado como no ejercitable en esta instancia. La consigna pide definir la política de retención, no demostrarla. Lo que sí se puede demostrar en la segunda entrega son las rutas, los tamaños y la promoción entre zonas.

"¿Por qué mantener Gold en Parquet si ya está en Cassandra?" Cassandra responde rápido las preguntas **conocidas**: las tablas se modelan desde la consulta. No responde las que todavía no se modelaron. Gold en Parquet queda como capa de exploración ad-hoc, que es donde se descubren las preguntas nuevas antes de convertirlas en tablas.

"¿last_reconciled_at no es sobrecarga?" Es la única forma de que el consumo distinga un valor provisorio del streaming de uno consolidado por la reconciliación. Sin esa marca, el mismo mart puede devolver dos valores distintos sin que nadie sepa cuál mirar. Es el costo declarado en la §6.7 y esta es su mitigación.

Defensa · §9 Flujos batch y streaming

Qué se decidió acá

#	Decisión	Alternativa descartada
1	Watermark de 7 días, aceptando que descarta el 91% de los eventos	Watermark de 60 días que cubra el span completo
2	<code>foreachBatch</code> con función de transformación compartida	Dos implementaciones separadas para streaming y batch
3	<code>maxFilesPerTrigger = 1</code>	Dejar que Spark agrupe archivos por defecto
4	Quarantine para lo que impide calcular, flag para lo sospechoso	Rechazar todo lo que viole una regla
5	<code>Joins left</code> contra la dimensión, nunca <code>inner</code>	<code>inner</code> por simplicidad

1. El número que hay que saber de memoria

Es el dato más fuerte del documento entero y conviene poder decirlo sin leer.

El mecanismo. Structured Streaming descarta de la agregación las filas cuyo event-time es anterior a `max(event_time) - watermark`. Como cada archivo cubre el rango completo del dataset, **el primer micro-lote ya lleva el máximo a 2025-08-31**. Desde el segundo en adelante, todo lo anterior al umbral cae.

La tabla.

Watermark	% procesado	% descartado
1 día	2,5%	97,5%
7 días	8,9%	91,1%
30 días	51,9%	48,1%
60 días	100%	0%

La frase. Incluso con un watermark de **30 días se descartaría casi la mitad de los eventos**. No es un bug: es la consecuencia directa de que la fragmentación sea aleatoria y no temporal.

Por qué esto es bueno para nosotros. Es la validación empírica de la decisión de patrón de la §6. La reconciliación no es una precaución teórica: con estos datos es **lo único que hace correctos a los marts**. Pocos equipos van a poder cuantificar por qué eligieron su patrón.

2. Por qué 7 días y no 60

La pregunta obvia. Si con 60 días procesamos el 100%, ¿por qué no 60?

La respuesta. Porque el watermark se dimensiona contra el caso productivo, no contra la muestra — es el principio 4 de la §8.1 y lo venimos sosteniendo desde la §3.

El estado de deduplicación crece con la ventana. En el escenario mediano de la §3.2 (3,6 millones de eventos diarios):

- 7 días → ~25 millones de claves en estado, del orden de cientos de MB. Viable.
- 60 días → más de 200 millones de claves, varios GB. Inviabile en un solo nodo.

Si insisten: "pero en la demo van a procesar el 9%". Sí, y la demo va a mostrar exactamente por qué existe la reconciliación. Elegir 60 días para que la demo quede linda sería diseñar para la demo, que es justo lo que venimos evitando. La métrica de filas descartadas por watermark está en la §9.9 precisamente para que ese efecto sea visible y no quede escondido.

3. foreachBatch y la lógica compartida

El problema que resuelve. En la §6.7 declaramos un costo del patrón híbrido: la agregación de `org_daily_usage_by_service` se implementa dos veces, en streaming y en batch. Es el costo que define a Lambda y la razón por la que Kappa existe.

La mitigación. `foreachBatch` entrega cada micro-lote como un DataFrame estático. Eso permite invocar **la misma función** `transformar_eventos(df)` desde el flujo C y desde el flujo D. Hay dos rutas, pero una sola implementación de la lógica de negocio.

El encuadre. Es el patrón `extract / transform / validate / load` de la Clase 03 aplicado donde realmente rinde: la claridad de responsabilidades vale más que una celda mágica.

Si preguntan "¿entonces no hay duplicación?". Hay duplicación de **orquestración** —dos jobs que disparar, dos configuraciones— pero no de lógica de transformación. Es una reducción del costo, no su eliminación, y así está declarado.

4. Quarantine frente a flag: el criterio

La regla en una línea. Se rechaza lo que **impide calcular**; se marca lo que resulta **sospechoso pero calculable**.

Aplicado:

- `value` no casteable a `double` → impide calcular la feature → **quarantine**.
- `unit` nulo con `value` presente → la métrica es ininterpretable → **quarantine**.
- `cost_usd_increment` de $-19,82$ → se calcula perfectamente y puede ser una nota de crédito legítima → **flag**.
- `last_login` anterior a `created_at` → el dato sirve igual para contar usuarios activos → **flag**.

Si preguntan "¿por qué no rechazar todo lo que viole una regla?". Porque se perdería información recuperable. Un costo negativo rechazado es `revenue` que desaparece del `mart` sin que nadie se entere. Marcado, `FinOps` decide qué hacer con él.

5. Por qué los joins son `left`

La evidencia. 20 de 80 organizaciones no tienen encuesta NPS. 2 de 80 no tienen eventos en la muestra.

La consecuencia de un `inner`. Esas organizaciones desaparecerían de los `mart`s **en silencio**. El `mart` seguiría funcionando, los conteos seguirían cerrando entre sí, y nadie notaría que faltan clientes hasta que alguien pregunte por uno en particular.

Es el tipo de error que no rompe nada, y por eso es peligroso.

Preguntas incómodas previstas

"¿Por qué `maxFilesPerTrigger = 1`?" Porque la fragmentación en ~ 20 archivos es intencional y simula micro-lotes. Un archivo por `trigger` reproduce esa intención. Si `Spark` agrupara varios archivos por lote, se perdería la granularidad que la consigna busca ejercitar.

"El `dropDuplicates` sobre streaming, ¿no acumula estado infinito?" No, porque está combinado con el `watermark`: `Spark` libera las claves más viejas que la ventana. Sin `watermark` el estado sería efectivamente ilimitado, y es el error clásico de esta construcción.

"¿Qué pasa con los eventos que el `watermark` descarta? ¿Se pierden?" No. El `watermark` los excluye de la **agregación en tiempo real**, no de `Bronze`. `Bronze` persiste todos los eventos, tardíos incluidos, y el flujo D los recupera. La distinción entre "descartado de la agregación" y "perdido" es central.

"¿Cada cuánto corre la reconciliación?" Diaria como punto de partida. El criterio es que la ventana de inconsistencia —entre la corrida de streaming y la de reconciliación— sea tolerable para `FinOps`. El `mart` expone `last_reconciled_at` para que el consumo sepa qué está mirando.

"Si streaming procesa el 9% y batch el 100%, ¿para qué sirve streaming?" Para el 9% que importa ahora. Una anomalía de costo de hoy se detecta en minutos; la corrección del histórico puede esperar al ciclo diario. Es exactamente la división de responsabilidades que O1 pide: frescura sobre lo reciente, completitud sobre el conjunto.

Defensa · \$10 Lógica MapReduce

Qué se decidió acá

#	Decisión	Alternativa descartada
1	Expresar <code>org_daily_usage_by_service</code>	Un job genérico de conteo
2	Particionar por <code>org_id</code> , no por la clave completa	Hash de la tupla completa
3	Emitir <code>n_eventos</code> desde el mapper	Calcular promedios en el reducer
4	Trazar con registros reales del dataset	Un ejemplo inventado
5	Mostrar el contraste con el <code>mart</code> de anomalías (3 pases)	Quedarse en el job simple

1. Por qué este job y no otro

El argumento. `org_daily_usage_by_service` es el `mart` que la consigna exige como mínimo en la segunda entrega, responde P1 y P2, y es el único que se calcula por las dos rutas del patrón híbrido. Expresarlo en MapReduce conecta la \$10 con la \$6, la \$7 y la \$9 en lugar de ser un ejercicio suelto.

Si preguntan "¿por qué no un `word count`?". Porque no demostraría nada del caso. La consigna pide el flujo batch **del caso**, no un ejemplo canónico.

2. El combiner y la condición que lo habilita

Esta es la parte que distingue entender el modelo de haberlo leído.

La afirmación. El combiner es válido **porque todas las agregaciones son sumas**, que son asociativas y conmutativas.

La consecuencia si no lo fueran. Si el `mart` necesitara un promedio, el combiner **no podría emitir `avg`**: el promedio de promedios no es el promedio. Habría que emitir (`suma`, `conteo`) y dividir recién en el reducer.

Por eso `n_eventos` **viaja desde el mapper**. No es decorativo: deja el `mart` preparado para derivar promedios sin romper la asociatividad.

Si preguntan "¿en `Spark` hay que escribir el combiner?". No: `Catalyst` aplica la agregación parcial antes del `Exchange` automáticamente. Es una de las diferencias de la \$10.8 — en MapReduce la verificación de asociatividad es responsabilidad del programador.

3. Por qué particionamos por `org_id`

El argumento. Particionar por la clave completa (`org_id`, `fecha`, `service`) distribuiría mejor, pero dispersaría los datos de una misma organización entre todos los reducers.

Lo que ganamos. Todas las fechas y servicios de una organización caen en el mismo reducer. Eso habilita que el job encadenado de anomalías calcule estadísticas por organización **sin un segundo shuffle**.

El riesgo que asumimos. `Skew`: si una organización concentrara un volumen desproporcionado, su reducer sería el cuello de botella. En el dataset la distribución es pareja (78 organizaciones con eventos), pero a escala productiva las cuentas enterprise podrían desbalancear. La mitigación es salt en la clave y un segundo pase de agregación.

Si preguntan "¿por qué no elegir siempre la distribución más pareja?". Porque el particionado es una decisión de colocación, no solo de balance. Colocar

datos que se van a usar juntos ahorra un shuffle entero después.

4. El hallazgo que salió del trazado

Esto es lo más importante de la sección, y vale la pena contarlo como lo que fue: **trabajar el ejemplo con datos reales nos obligó a corregir una regla de la §9.7.**

La tensión. La §9.7 decía que un `value` no casteable manda el registro a quarantine. Pero al trazar `evt_gh4g7q9rxnqk` apareció que ese evento tiene `value` nulo y `cost_usd_increment = 1,0336`, que es perfectamente válido. Descartar el registro entero perdería ese costo.

La cuantificación. Los 9 eventos con `value` nulo y los 17 con `unit` nulo arrastran **el 4,6% del costo total** del archivo (43,35 de 932,82 USD).

La corrección (D-009). El rechazo opera sobre el **campo derivado**, no sobre el registro. El evento promueve con la métrica en nulo y marcada, el costo participa del mart, y una copia va a quarantine para auditoría.

En el trazado se ve. La celda de `org_cvs4f8cg` el 2025-08-23 da 2,4002 USD. Si se hubiera descartado el registro defectuoso daría 1,3666: **un 43% menos en esa celda.**

Por qué contarlo así. Es coherente con D-007: un error que subestima el revenue por organización es exactamente la clase de error que venimos persiguiendo. Y muestra que el trazado no fue un trámite: encontró algo.

Consecuencia conceptual. Quarantine deja de ser "lo que no entró" y pasa a ser "lo que entró con reservas, con su motivo". El conteo de quarantine ya no es complementario al de Silver, y eso hay que saber explicarlo si alguien suma las dos cifras y no le cierra.

5. El contraste que justifica usar Spark

El caso. `cost_anomaly_mart` necesita tres pasos: agregar el costo diario, calcular la estadística de referencia por organización y servicio, y marcar cada día contra ella.

En MapReduce: tres jobs encadenados, y entre cada uno el intermedio se escribe a HDFS **con replicación triple** y se vuelve a leer, más el arranque de JVM por tarea.

En Spark: un solo DAG, el intermedio cacheado en memoria, y —gracias al particionado por `org_id`— sin shuffle adicional en el paso 2.

La frase que cierra. No es que MapReduce no pueda resolver el caso: lo resuelve pagando I/O innecesario en cada eslabón de una cadena que acá tiene tres.

Preguntas incómodas previstas

"¿Por qué el mapper proyecta la métrica a columnas?" La fuente está en formato largo (`metric / value`) y el mart lo necesita ancho. Al hacerlo en el map, el shuffle mueve tuplas ya proyectadas en lugar de filas crudas: menos bytes por la red. Es el equivalente conceptual del *predicate pushdown* que Catalyst hace solo.

"¿El combiner no puede cambiar el resultado?" No, siempre que la operación sea asociativa y conmutativa. Ese es exactamente el contrato: el framework puede invocarlo cero, una o muchas veces, y el resultado debe ser el mismo. Por eso no se puede promediar en el combiner.

"¿Cómo manejan v1 y v2 en el mapper?" `carbon_kg` solo se lee si `schema_version = 2`; `genai_tokens` solo si además `service = 'genai'`. Es la regla compuesta de la §4.3, y escribirla como "v2 implica tokens" produciría 251 falsos positivos sobre 275 eventos v2.

"¿Van a implementar MapReduce?" No. La sección demuestra que el procesamiento batch se entiende al nivel del modelo de programación y dónde Spark se aparta de él. La implementación es Spark, y la §10.9 explica por qué con un caso concreto del propio proyecto.

Defensa · §11 Supuestos, riesgos y decisiones abiertas

Qué se decidió acá

#	Decisión	Alternativa descartada
1	MAD sobre la serie diaria por (<code>org</code> , <code>service</code>)	z-score global, percentiles, MAD global
2	Detectar sobre el mart diario, no sobre eventos crudos	Detectar en Silver, evento por evento
3	Los negativos se tratan aparte de los picos	Un solo mecanismo para todo lo raro
4	Cada supuesto se declara con su consecuencia si es falso	Listar supuestos sin impacto
5	Dejar tres decisiones abiertas en lugar de forzar el cierre	Cerrar todo para que el documento se vea terminado

1. El argumento del método de anomalías

Es la parte más fuerte de la sección porque la conclusión sale de haber probado los tres métodos, no de haber elegido el que suena mejor.

Paso 1 — Los tres fallan aplicados globalmente.

Método	Marcados
z-score $ z > 3$	1,7%
MAD $ mz > 3,5$	23,9%
Percentiles p01/p99	1,9%

Paso 2 — El diagnóstico. Que MAD marque casi una cuarta parte no es un defecto del método: es un síntoma. La distribución global es **una mezcla de escalas incompatibles**.

Métrica	Mediana de costo
<code>cpu_hours</code>	0,0594
<code>storage_gb_hours</code>	0,3515
<code>requests</code>	5,1741

Un evento de `requests` cuesta 87 veces la mediana de uno de `cpu_hours`. Cualquier estadística global marca la métrica cara por ser cara.

Paso 3 — Por qué agrupar por servicio tampoco sirve. Es el intento intuitivo y falla: cada servicio mezcla métricas, y sus medianas quedan artificialmente parejas entre 0,46 y 1,23. Probado: MAD por servicio marca el 26,1%, peor que el global.

Paso 4 — La solución. Detectar sobre el **mart diario**, donde la mezcla de métricas ya está colapsada en un `daily_cost_usd` por celda, con línea de base por serie (`org_id`, `service`) a lo largo del tiempo. Cada día se compara contra el comportamiento histórico de esa misma organización en ese mismo servicio.

Si preguntan "¿por qué MAD y no z-score?". Media y desvío se contaminan con los propios valores extremos que buscan detectar: un spike infla el desvío y se vuelve menos detectable. Mediana y MAD tienen punto de ruptura del 50%. Con media 2,59 contra mediana 0,75, la asimetría lo hace material.

Si preguntan "¿por qué no percentiles?". Un corte en p99 marca siempre el 1% de los casos, haya o no anomalías. Es una cuota, no una detección.

Si preguntan "¿tienen historia suficiente?". Verificado: 181 series (`org`, `service`); extrapoladas a los ~20 archivos, todas superarían los 20 días. Con menos de 20 días se cae a la estadística por servicio y se marca el origen de la línea de base.

2. Por qué los negativos van aparte

El argumento. Un costo de −19,82 no es un spike de consumo: es probablemente una nota de crédito. Es un fenómeno de negocio distinto.

El problema de mezclarlos. Un negativo grande desplaza la línea de base de la serie y vuelve menos detectables los picos reales. Se marca con `is_credit_adjustment` y no compite con la detección de spikes.

Si preguntan "¿y si es un error de facturación?". También interesa, pero es otra alerta con otro destinatario: un ajuste de crédito lo revisa Finanzas, un spike de consumo lo revisa Customer Success. Dos señales, dos flags.

3. El límite que reconocemos

Lo que declaramos. Con 4 meses de datos no hay estacionalidad observable. Un método que descompusiera tendencia y estacionalidad sería superior a MAD, pero **no se puede calibrar ni validar con esta ventana**.

Por qué decirlo. Si alguien pregunta por qué no usamos algo más sofisticado, la respuesta ya está escrita y es una razón, no una excusa. Y queda como próximo paso en la §12.

4. Los supuestos y su consecuencia

El criterio de redacción. Cada supuesto se declara con qué pasaría si fuera falso. Un supuesto sin consecuencia declarada no es un supuesto: es una afirmación disfrazada.

Los dos que más conviene tener presentes:

- **S-02** (el dataset es una réplica reducida): si fuera falso, se cae la justificación de Big Data entera y el proyecto sería un ejercicio de pandas. Es el supuesto sobre el que descansa la §3.
- **S-05** (el FX ruidoso en USD es un defecto y no una moneda mal etiquetada): si fuera falso se revierte D-007. La evidencia a favor es que las tasas de USD se distribuyen **simétricamente alrededor de 1,0**; si la etiqueta estuviera mal, se parecerían a las de ARS o EUR.

5. Por qué dejamos tres decisiones abiertas

La tentación. Cerrar todo para que el documento se vea terminado.

Por qué no. La consigna pide explícitamente "decisiones todavía abiertas". Y una decisión cerrada sin evidencia es peor que una abierta con fecha: en noviembre hay que reabrirla igual, pero con código escrito encima.

Las tres, y por qué cada una sigue abierta:

- **A-07** (orden cronológico en los ~20 archivos): es la única que condiciona una decisión ya tomada. Si los eventos llegaron ordenados, cae el fundamento de D-008 y posiblemente de D-004. Depende de recibir el dataset completo.
- **A-08** (claves de Cassandra): se cierra contrastando el CQL contra las cinco consultas. Decidirlo en papel sin ejecutarlo sería adivinar.
- **A-09** (alcance del ML): depende de evaluar la señal disponible, y el perfilado ya sugiere que es débil.

Si preguntan "¿no es un riesgo dejarlas abiertas?". Lo riesgoso es lo contrario. A-07 está anotada precisamente para que, si el dataset completo contradice el supuesto, la revisión sea una decisión planificada y no un descubrimiento.

Preguntas incómodas previstas

"G-01 dice que podrían generar datos sintéticos. ¿No es hacer trampa?" No reemplazaría al dataset provisto: sería un plan de contingencia para poder ejercitar el streaming si los ~20 archivos no llegan. El generador replicaría el esquema v1/v2 y la fragmentación observada, y quedaría declarado como tal en las evidencias.

"¿Por qué G-03 y G-05 tienen probabilidad alta y siguen sin resolverse?" Porque su impacto es medio y sus mitigaciones son de ejecución, no de diseño. G-03 se mitiga con un Quickstart validado, y G-05 ya está mitigado por construcción: P7 se declaró deseable desde la §2.3.

"El riesgo G-04 (divergencia entre rutas) es el costo que ustedes mismos eligieron en \$6." Correcto, y por eso está listado. La mitigación es la función compartida vía `foreachBatch` más una prueba que compare las salidas de ambas rutas sobre el mismo período. Un costo aceptado tiene que quedar monitoreado, no olvidado.

"¿Cómo saben que el umbral 3,5 es el correcto?" Es el valor convencional para el z-score modificado de Iglewicz y Hoaglin. No está calibrado contra este dataset porque no hay etiquetas de anomalía verdadera. Se ajustará en la segunda entrega observando la tasa de marcado por serie, y ese ajuste quedará registrado.

Defensa · §12 Esfuerzo, roles y recursos

Qué se decidió acá

#	Decisión	Alternativa descartada
1	Roles por capacidad end-to-end, no por capa del pipeline	Uno por zona: Bronze, Silver, Gold, Serving
2	Cada rol con suplente nombrado	Cuatro especialistas sin redundancia
3	Estimar en horas-persona por rubro, no por fecha	Un cronograma de tareas con fechas
4	Declarar el pico de la instancia final	Repartir el esfuerzo de forma pareja en el papel

1. Por qué los roles son por capacidad y no por capa

El reparto intuitivo. Uno hace Bronze, otro Silver, otro Gold, otro Serving. Es lo que sale solo con cuatro personas y un pipeline de cuatro zonas.

Por qué no. Genera exactamente el problema que el pipeline ya tiene: las fronteras. ¿De quién es la regla de calidad que se aplica al promover de Bronze a Silver? ¿Quién escribe la prueba que compara la salida de streaming contra la de la reconciliación? En un reparto por capas, las fronteras no son de nadie — y las fronteras son donde vive toda la §9.7.

Lo que hicimos. R2 no entrega "Bronze": entrega **dato ingerido y validado**, con sus reglas y sus pruebas. La capacidad cruza la frontera, y el responsable también.

Si preguntan "¿no se pisan entre roles?". Se tocan en puntos definidos: R2 entrega a R3 un contrato de datos, R3 entrega a R4 marts con grano declarado. El contrato es el diccionario de datos, que es propiedad de R1.

2. Por qué cada rol tiene suplente

El riesgo real de un equipo de 4. No es la falta de manos: es que una persona sea el único que entiende el streaming la semana antes de la entrega.

La regla. Cada rol tiene un suplente nombrado, y la revisión cruzada obligatoria antes de cada entrega es precisamente entre titular y suplente. Así la redundancia no es teórica: se ejercita en cada hito.

Efecto colateral buscado. La consigna pide que la defensa oral tenga participación de todos los integrantes. Con suplencias ejercitadas, cualquiera puede responder por dos áreas y no solo por la propia.

3. Sobre las estimaciones

Por qué horas-persona por rubro y no un cronograma con fechas. Un cronograma detallado a 11 semanas se desactualiza en la primera. Las horas por rubro se mantienen útiles aunque el orden cambie, y permiten repriorizar el backlog sin rehacer el plan.

Los números que hay que poder defender:

- **Streaming, 24 h:** es el rubro más caro de la segunda entrega porque concentra watermark, deduplicación inter-lote con estado, late data y checkpointing — todos los problemas que el perfilado documentó en la §3.3.
- **Silver, 24 h:** compatibilidad v1/v2 más conformación de siete fuentes con granos distintos.
- **Presentación y video, 20 h:** se subestima siempre y es requisito explícito de la instancia final.

El total: 286 horas-persona. Sobre 4 integrantes y 11 semanas da unas 6,5 horas semanales por persona, compatible con una materia electiva.

4. El pico que declaramos

El dato. La instancia final concentra 102 horas en 3 semanas: unas 8,5 semanales por integrante, contra 5 en el tramo de la segunda entrega.

Por qué lo decimos. Porque es información para priorizar el backlog en noviembre, no una queja. Si al 16/11 quedan marts opcionales pendientes, el pico se vuelve inviable — y eso es exactamente el riesgo G-07.

Si preguntan "¿por qué no repartirlo mejor?". No se puede: la presentación, el video y el ensayo de la defensa solo se pueden hacer sobre una solución que ya funciona. La alternativa sería adelantar trabajo técnico a la segunda entrega, que es lo que el backlog priorizado debe resolver.

Preguntas incómodas previstas

"¿Cómo estimaron si nunca hicieron esto?" Sobre la base del perfilado: sabemos cuántas fuentes hay, qué problemas tiene cada una y qué reglas hacen falta. Los rubros caros son los que el perfilado mostró complicados. Es una estimación fundada en evidencia, no un promedio inventado — y como toda estimación, se corrige con lo aprendido en la segunda entrega.

"El trabajo restante de esta entrega incluye 'obtener los ~20 archivos'. ¿Entregan sin eso?" Sí, y está declarado. La primera entrega es de diseño y el perfilado sobre un archivo fue suficiente para fundamentarlo. Lo que el archivo faltante bloquea es A-07, que es una verificación de un supuesto, no un entregable de esta instancia. El riesgo G-01 tiene su mitigación: un generador sintético si no llegan.

"¿Por qué R1 posee el documento de diseño si lo escriben entre todos?" Poseer no es escribir: es responder por la coherencia entre lo que el documento dice y lo que el repositorio hace. Es el criterio de aceptación de la segunda entrega —que el diagrama represente lo implementado— y necesita un responsable único.

"¿Qué pasa si alguien abandona la materia?" Las suplencias cubren el conocimiento, pero no las horas. Con tres integrantes el total por persona pasaría de 6,5 a 8,7 horas semanales, y habría que recortar por el backlog: primero lo declarado deseable, después P7.